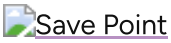
Save Point

Untitled

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026



QuillFin

A mobile-first notes app built around spaced repetition: notes, AI-generated quizzes and practice problems, scheduled with FSRS.
Built and owned by **Save Point Limited**, New Zealand.

Docs · Running Locally · Plan · Testing · Web Build · Security · Licence

---

> [!IMPORTANT] QuillFin is in development and is Save Point's proprietary software. It is **not** open source. See LICENSE.

What it does

Keep notes in subject folders, turn them into quiz questions with your own AI account, track practice problems LeetCode-style, and let the FSRS forgetting curve bring each one back right before you'd forget it.

- 🧠 **FSRS scheduling.** The algorithm Anki uses, for every quiz question and problem. Answers are auto-graded and then rated Again/Hard/Good/Easy, each button showing when that rating brings the item back. Target recall is adjustable. See scheduling_algorithm.md.
- 🤖 **Bring your own AI key.** Connect OpenAI (or any OpenAI-compatible endpoint: OpenRouter, Groq, a local Ollama or LM Studio server) or Anthropic from the Account tab. The app itself is never billed.
- 📥 **Import your own files.** `.txt`, `.md`, `.pdf`, `.docx` and `.pptx`, single or in bulk, with a preview and reorder step before anything is imported.
- 🧩 **Problem tracker.** Log practice problems with difficulty, tags, link and your solution, time each attempt, and re-solve them on their own FSRS schedule.
- 📅 **Today dashboard and reminders.** What's due, reviews done today, average recall and a 7-day forecast, plus an optional daily study notification.
- ☁️ **Optional accounts and sync.** Sign in to sync notes, questions and problems across devices, offline-first. Or stay on this device with no account at all. See accounts_and_sync.md.
- 📱 **Works at every size.** A bottom bar on phones, a rail on tablets, and a sidebar, list-detail panes, a table view and keyboard shortcuts on desktop. Light and dark themes.

QuillFin is moving to accounts (Clerk), sync (Convex) and a Pro plan (Stripe), with a Next.js web app alongside this Flutter app. The Convex backend is in `backend/` and the web app is in `webapp/`.

This is a from-scratch Flutter rewrite of an earlier Python/Tkinter desktop app, `Quiz-app-Ai`. It reuses that project's ideas and, where useful, its algorithms ported to Dart, but not its code.

Quick start (Save Point staff)

Requires the Flutter SDK (built against 3.47.2) and a connected device, a running emulator, or `flutter build apk` for a standalone install.

```
flutter pub get
flutter run
```

See running_locally.md for full setup, first-run notes and where data lives on the device.

Testing

```
flutter analyze
flutter test
flutter test --coverage && dart run tool/check_coverage.dart   # checks 100% line coverage of lib/
flutter test integration_test -d linux                         # end-to-end on Linux desktop
```

Suite	Tests	Coverage
Flutter app	394, from pure logic to widget tests for every screen	100% of `lib/`
Convex backend (Vitest, convex-test)	131, including FSRS parity with the app	100% of `backend/convex/`
Web app (Vitest, Testing Library)	31	

The end-to-end test runs the real app on Linux desktop (real SQLite file, settings and keychain, real HTTP to a local fake AI server), from sign-in through quiz, problem tracking, import and export. The backend and web suites run from the repo root with Node 24:

```
npm install
npm run backend:test
npm run web:test
```

testing.md explains why widget tests use `tester.tap()` rather than simulated device input, and the ADB-tap lesson behind that.

Project layout

```
quillfin/
├── lib/                    # Flutter app (Android, iOS, desktop, web)
│   ├── main.dart  app.dart
│   ├── screens/  layout/  theme/        # UI
│   ├── core/               # FSRS scheduling, grading, models
│   ├── storage/            # Local SQLite database
│   ├── sync/  auth/        # Offline-first sync with Convex, Clerk sign-in
│   ├── ai/                 # AI providers (your own OpenAI / Anthropic key)
│   └── config/  services/  settings/
├── backend/                # Convex backend: sync protocol, plans, AI generation, billing
│   ├── convex/
│   └── tests/
├── webapp/                 # Next.js web app
│   ├── app/  components/  lib/  public/  test/
├── test/  integration_test/   # Flutter unit, widget and integration tests
├── tool/                   # build_docs.dart, check_coverage.dart, FSRS parity fixture
├── config/example.json     # Build config template (real config/*.json is git-ignored)
├── Docs/                   # Documentation, plus branded PDFs in Docs/pdf/
├── docker/  Dockerfile  docker-compose.yml   # Web build behind nginx
├── .github/workflows/      # backend.yml, webapp.yml, build.yml (release installers)
├── android/  ios/  linux/  macos/  windows/  web/   # Flutter platform runners
├── pubspec.yaml
├── LICENSE                 # Proprietary, all rights reserved
└── SECURITY.md
```

Secrets

No keys live in this repo. Server-side keys (`ANTHROPIC_API_KEY` , `STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` , Clerk settings) are set on the Convex deployment with `npx convex env set` ; see backend.md. Local settings go in `backend/.env.local` , `webapp/.env.local` (from `webapp/.env.example`) and `config/*.json` (from `config/example.json`), all git-ignored, as are Android keystores and `key.properties` . A user's own AI key is kept in the device's secure storage and sent only to their AI provider, never to Save Point. See SECURITY.md.

Documentation

Everything needed to understand and extend this codebase is in **Docs/**. Save Point branded PDFs are in `Docs/pdf/` , rebuilt with `dart run tool/build_docs.dart` .

Doc	What's in it
architecture.md	Module map, layering, and how this relates to the original Python app
database_schema.md	The SQLite schema
scheduling_algorithm.md	FSRS scheduling, ratings and the v1 migration
ai_providers.md	The AI provider abstraction and how to add a third
accounts_and_sync.md	Signing in, sync columns and triggers, the sync engine, build config
webapp.md	The Next.js web app: screens, layouts, shared backend code, tests
backend.md	The Convex backend: sync protocol, plans, AI generation, billing, env vars
setup_guide.md	Creating the Convex, Clerk, Stripe and Anthropic accounts and keys
store_listing.md	Google Play and App Store submission: listing, privacy answers, signing, and the plan for iPhone
quillfin_plan.md	The production plan: architecture, sync, pricing, layouts, phases
gui_theming.md	The (placeholder) QuillFin brand tokens and theme
testing.md	Running tests, coverage, fakes and patterns, what's not covered
running_locally.md	Setup, first run, where data lives, Android build gotchas
packaging.md	Windows installer and Linux AppImage, built on each GitHub release
hosting.md	The web build in Docker behind a reverse proxy at quillfin.savepoint.co.nz
future_work.md	Deliberate deferrals and what's not built yet

Ownership

QuillFin is © 2026 Save Point Limited, a New Zealand company (company number 9392145). All rights reserved.

- Website: savepoint.co.nz
- General enquiries: contact@savepoint.co.nz
- Support, privacy and security: support@savepoint.co.nz

"Save Point" and the Save Point flag logo are the property of Save Point Limited. OpenAI, Anthropic, Anki, Clerk, Convex and Stripe are trademarks of their respective owners. This project is not affiliated with or endorsed by any of them.